

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN



HCMUTE

MÔN HỌC: MẬT MÃ ỨNG DỤNG

BÁO CÁO CUỐI KÌ

**NGHIÊN CỨU CÁC THUẬT TOÁN MẬT MÃ ĐƯỢC ỨNG
DỤNG TRONG HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU
MICROSOFT SQL SERVER**

GVHD: Trần Đức Tốt

SVTH:

Nguyễn Vĩnh An 23162002

Phạm Gia Bảo 23162007

Lê Thanh Sơn 23162084

Nguyễn Văn Tâm 23162088

Mã lớp học: 242CRYP331980_02

Thành phố Hồ Chí Minh, Tháng 5 năm 2025

MỤC LỤC

PHẦN MỞ ĐẦU.....	1
1. Tính cấp thiết của đề tài.....	1
2. Mục tiêu nghiên cứu.....	1
3. Đối tượng, phạm vi nghiên cứu.....	2
CHƯƠNG 1: TỔNG QUAN VỀ MẬT MÃ VÀ AN TOÀN DỮ LIỆU TRONG SQL SERVER.....	3
1.1 GIỚI THIỆU VỀ MẬT MÃ.....	3
1.1.1 Định nghĩa và tầm quan trọng của mật mã trong bảo vệ dữ liệu.....	3
1.1.2 Các khái niệm cơ bản.....	3
1.1.3 Phân loại các thuật toán mật mã.....	3
1.2. AN TOÀN DỮ LIỆU VÀ SQL SERVER.....	4
1.2.1. Các mối đe dọa an ninh trong cơ sở dữ liệu.....	4
1.2.2. Các phương pháp bảo vệ dữ liệu trong SQL Server.....	4
1.2.3. Vai trò của mã hóa trong chiến lược bảo mật của SQL Server.....	5
1.3 CÁC TÍNH NĂNG MẬT MÃ TÍCH HỢP TRONG SQL SERVER.....	5
1.3.1 Tổng quan về tính năng mã hóa trong SQL Server.....	5
1.3.2 So sánh các phương pháp mã hóa.....	5
1.3.3 Trường hợp sử dụng phù hợp của các phương pháp mã hóa.....	6
CHƯƠNG 2: NGHIÊN CỨU CHI TIẾT CÁC THUẬT TOÁN MẬT MÃ TRONG SQL SERVER.....	6
2.1. MÃ HÓA ĐỐI XỨNG (SYMMETRIC ENCRYPTION).....	7
2.1.1 Giới thiệu và Nguyên tắc hoạt động.....	7
2.1.2 Các thuật toán đối xứng phổ biến trong SQL Server.....	7
2.1.3 Ưu và nhược điểm của mã hóa đối xứng trong SQL Server.....	8
2.2. MÃ HÓA BẤT ĐỐI XỨNG (ASYMMETRIC ENCRYPTION).....	8
2.2.1 Giới thiệu và nguyên tắc hoạt động.....	8
2.2.2 Các thuật toán bất đối xứng trong SQL Server.....	9
2.2.3 Ưu và nhược điểm của mã hóa bất đối xứng trong SQL Server.....	9
2.3 HÀM BẮM (CRYPTOGRAPHIC HASH FUNCTIONS).....	10
2.3.1 Giới thiệu và thuộc tính.....	10
2.3.2 Các thuật toán băm phổ biến.....	10
2.3.3 Ứng dụng của hàm băm trong SQL Server.....	10
2.4 QUẢN LÝ KHÓA MÃ HÓA TRONG SQL SERVER.....	10
2.4.1 Tầm quan trọng.....	11
2.4.2 Các phương pháp quản lý khóa.....	11
2.4.3 Các nguyên tắc và thực hành tốt nhất trong quản lý khóa.....	11
CHƯƠNG 3: XÂY DỰNG CHƯƠNG TRÌNH ỨNG DỤNG CÁC THUẬT TOÁN MẬT MÃ TRONG SQL SERVER.....	11
3.1 LỰA CHỌN NGÔN NGỮ LẬP TRÌNH VÀ CÔNG CỤ.....	12
3.2 THIẾT KẾ CƠ SỞ DỮ LIỆU CHO ỨNG DỤNG.....	13
3.3 TRIỂN KHAI MÃ HÓA VÀ GIẢI MÃ DỮ LIỆU.....	14

3.4 ỨNG DỤNG HÀM BẮM ĐỂ LƯU TRỮ MẬT KHẨU AN TOÀN	15
3.5 QUẢN LÝ KHÓA TRONG ỨNG DỤNG	16
3.6 XÂY DỰNG GIAO DIỆN NGƯỜI DÙNG	17
CHƯƠNG 4: KIỂM THỬ, KHẮC PHỤC LỖI VÀ ĐÁNH GIÁ ỨNG DỤNG	20
4.1 KIỂM THỬ	21
4.1.1 Kiểm thử chức năng	21
4.1.2 Kiểm thử bảo mật	35
4.2 KHẮC PHỤC LỖI.....	39
4.2.1 Phân tích lỗi.....	39
4.2.2 Sửa lỗi	39
4.2.3 Kiểm tra lại.....	40
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	44
5.1 TÓM TẮT KẾT QUẢ NGHIÊN CỨU VÀ PHÁT TRIỂN ỨNG DỤNG	44
5.2 THẢO LUẬN VỀ NHỮNG THÁCH THỨC VÀ BÀI HỌC KINH NGHIỆM TRONG QUÁ TRÌNH THỰC HÀNH.....	44
5.3 ĐỀ XUẤT CÁC HƯỚNG PHÁT TRIỂN TIẾP THEO CHO ỨNG DỤNG..	45
TÀI LIỆU THAM KHẢO	47

PHẦN MỞ ĐẦU

1. Tính cấp thiết của đề tài

Trong thời đại chuyển đổi số mạnh mẽ hiện nay, dữ liệu đóng vai trò là tài sản cốt lõi và quan trọng nhất của mỗi tổ chức, doanh nghiệp. Việc bảo vệ an toàn thông tin, đặc biệt là dữ liệu lưu trữ trong hệ thống cơ sở dữ liệu, trở thành yêu cầu bắt buộc nhằm đảm bảo tính bí mật, toàn vẹn và sẵn sàng của dữ liệu. Các cuộc tấn công mạng ngày càng tinh vi và có chủ đích, nhắm trực tiếp vào dữ liệu người dùng, tài chính, thông tin khách hàng,... khiến nhu cầu về bảo mật dữ liệu ngày càng trở nên cấp bách hơn bao giờ hết.

Microsoft SQL Server – một trong những hệ quản trị cơ sở dữ liệu phổ biến và được sử dụng rộng rãi – đã tích hợp nhiều cơ chế và thuật toán mã hóa nhằm đảm bảo an toàn thông tin. Tuy nhiên, việc hiểu rõ nguyên lý hoạt động, đặc điểm, ưu – nhược điểm cũng như phạm vi ứng dụng của các thuật toán mật mã trong SQL Server vẫn còn là một thách thức đối với nhiều nhà quản trị hệ thống và lập trình viên cơ sở dữ liệu.

Do đó, việc nghiên cứu các thuật toán mật mã được ứng dụng trong SQL Server không chỉ giúp tăng cường nhận thức về bảo mật dữ liệu mà còn góp phần nâng cao hiệu quả triển khai, vận hành và bảo vệ cơ sở dữ liệu trong thực tiễn. Đề tài này mang tính thời sự, có giá trị lý luận và ứng dụng cao, đặc biệt trong bối cảnh các tổ chức đang chuyển dần sang mô hình quản trị dữ liệu tập trung và triển khai các giải pháp bảo mật tiên tiến nhằm tuân thủ các quy định pháp luật về bảo vệ dữ liệu cá nhân, chẳng hạn như GDPR hay Luật An ninh mạng.

2. Mục tiêu nghiên cứu

Mục tiêu đề tài tập trung vào việc khảo sát và hệ thống hóa các thuật toán mã hóa được tích hợp sẵn trong hệ quản trị cơ sở dữ liệu Microsoft SQL Server. Đồng thời, đề tài sẽ phân tích một cách chi tiết cơ chế hoạt động, ưu điểm và nhược điểm của từng loại thuật toán mã hóa, bao gồm mã hóa dữ liệu tĩnh (Data at Rest), dữ liệu truyền (Data in Transit), và các thuật toán băm (Hashing). Bên cạnh đó, nghiên cứu này còn hướng dẫn triển khai các kỹ thuật mã hóa trong môi trường Microsoft SQL thực tế, nhằm nâng cao bảo mật dữ

liệu. Cuối cùng, đề tài sẽ đánh giá ảnh hưởng của các phương pháp mã hóa đến hiệu suất hệ thống và mức độ bảo vệ thông tin.

3. Đối tượng, phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là các thuật toán mật mã được tích hợp và sử dụng trong hệ quản trị cơ sở dữ liệu Microsoft SQL Server. Cụ thể, đề tài tập trung vào các cơ chế mã hóa dữ liệu, bảo vệ thông tin và xác thực người dùng trong SQL Server, bao gồm nhưng không giới hạn ở các kỹ thuật như:

- Mã hóa đối xứng (symmetric encryption),
- Mã hóa bất đối xứng (asymmetric encryption),
- Băm mật mã (hashing),
- Transparent Data Encryption (TDE),
- Always Encrypted

Phạm vi nghiên cứu:

- Đề tài tập trung vào phiên bản Microsoft SQL Server từ 2016 trở về sau, do đây là các phiên bản hỗ trợ đầy đủ hơn về tính năng bảo mật hiện đại như Always Encrypted, các thuật toán mã hóa nâng cao, và cải tiến trong quản lý khóa.
- Nghiên cứu chủ yếu giới hạn trong môi trường SQL Server cài đặt tại chỗ (on-premises) và không đi sâu vào các nền tảng đám mây (cloud-based SQL như Azure SQL).
- Đề tài không đi sâu vào phân tích chi tiết các khía cạnh toán học của từng thuật toán, mà tập trung vào ứng dụng thực tiễn, cách thức triển khai, cấu hình, và so sánh hiệu quả giữa các kỹ thuật mã hóa trong môi trường SQL Server.
- Các nội dung ngoài phạm vi bảo mật dữ liệu (như tối ưu truy vấn, cấu trúc lưu trữ...) sẽ không được đề cập sâu trong đề tài này.

CHƯƠNG 1: TỔNG QUAN VỀ MẬT MÃ VÀ AN TOÀN DỮ LIỆU TRONG SQL SERVER

1.1 GIỚI THIỆU VỀ MẬT MÃ

1.1.1 Định nghĩa và tầm quan trọng của mật mã trong bảo vệ dữ liệu

Mật mã học (Cryptography) là lĩnh vực nghiên cứu và ứng dụng các kỹ thuật nhằm bảo vệ thông tin bằng cách biến đổi dữ liệu gốc thành dạng không thể hiểu được đối với người không có quyền truy cập. Trong thời đại kỹ thuật số, nơi dữ liệu trở thành tài sản cốt lõi của tổ chức, mật mã đóng vai trò then chốt trong việc đảm bảo tính bảo mật (confidentiality), toàn vẹn (integrity) và xác thực (authentication) của thông tin.

Khi được triển khai hiệu quả, các kỹ thuật mật mã sẽ giúp ngăn chặn truy cập trái phép, bảo vệ thông tin nhạy cảm khỏi rò rỉ hoặc bị giả mạo, từ đó duy trì niềm tin của người dùng và tuân thủ các quy định pháp lý như GDPR, HIPAA, hay ISO/IEC 27001.

1.1.2 Các khái niệm cơ bản

- Mã hóa (Encryption): Quá trình chuyển đổi dữ liệu ở dạng rõ (plaintext) thành dạng mã hóa (ciphertext) nhằm bảo vệ dữ liệu khỏi bị truy cập trái phép.
- Giải mã (Decryption): Quá trình khôi phục dữ liệu từ ciphertext về dạng plaintext, sử dụng khóa giải mã.
- Khóa (Key): Thông tin bí mật dùng để điều khiển quá trình mã hóa và giải mã. Có thể là khóa đối xứng (cùng một khóa) hoặc khóa bất đối xứng (cặp khóa công khai – riêng tư).
- Thuật toán mật mã (Cryptographic algorithm): Tập hợp các bước logic và toán học để thực hiện mã hóa và giải mã.

1.1.3 Phân loại các thuật toán mật mã

- Thuật toán đối xứng (Symmetric Encryption): Dùng cùng một khóa cho

cả mã hóa và giải mã. Ví dụ: AES, DES, 3DES. Ưu điểm là nhanh và hiệu quả, nhưng việc chia sẻ khóa an toàn là một thách thức.

- Thuật toán bất đối xứng (Asymmetric Encryption): Dùng một cặp khóa: khóa công khai để mã hóa và khóa riêng tư để giải mã. Ví dụ: RSA, ECC. Thích hợp cho môi trường truyền thông không an toàn.
- Hàm băm (Hash Function): Biến đổi dữ liệu bất kỳ thành chuỗi có độ dài cố định. Không thể đảo ngược và thường dùng để kiểm tra toàn vẹn dữ liệu. Ví dụ: SHA-256, MD5 (MD5 hiện không còn được khuyến nghị sử dụng).

1.2. AN TOÀN DỮ LIỆU VÀ SQL SERVER

1.2.1. Các mối đe dọa an ninh trong cơ sở dữ liệu

Trong SQL Server và các hệ quản trị cơ sở dữ liệu (HQT CSDL) nói chung, dữ liệu thường xuyên đối mặt với các nguy cơ bảo mật như:

- Truy cập trái phép: Người dùng hoặc phần mềm không được phép có thể truy cập hoặc lấy cắp dữ liệu.
- Tấn công SQL Injection: Kẻ tấn công chèn mã SQL độc hại để truy vấn dữ liệu bất hợp pháp.
- Rò rỉ dữ liệu (Data leakage): Dữ liệu nhạy cảm bị lộ qua các bản sao lưu không mã hóa, nhật ký hệ thống hoặc lỗi cấu hình.
- Thay đổi dữ liệu không hợp lệ: Dữ liệu bị chỉnh sửa mà không có dấu vết, gây ảnh hưởng đến tính toàn vẹn.

1.2.2. Các phương pháp bảo vệ dữ liệu trong SQL Server

SQL Server cung cấp nhiều lớp bảo mật để bảo vệ dữ liệu, bao gồm:

- Kiểm soát truy cập (Access Control): Quản lý quyền truy cập chi tiết đến từng đối tượng (bảng, cột, thủ tục...). Sử dụng vai trò, quyền và chính sách bảo mật.
- Kiểm toán (Auditing): Theo dõi và ghi lại các hành vi truy cập và thao

tác dữ liệu, phục vụ cho phát hiện sự cố và tuân thủ.

- Mã hóa dữ liệu ở nhiều cấp độ như: cột, tệp, hoặc toàn bộ cơ sở dữ liệu để bảo vệ dữ liệu khi lưu trữ hoặc truyền tải.

1.2.3. Vai trò của mã hóa trong chiến lược bảo mật của SQL Server

Trong chiến lược bảo mật của SQL Server, mã hóa đóng vai trò như lá chắn cuối cùng. Khi các biện pháp như phân quyền hay firewall bị vượt qua, mã hóa đảm bảo rằng kẻ tấn công vẫn không thể đọc được dữ liệu nếu không có khóa phù hợp.

Ngoài ra, mã hóa cũng là một yêu cầu bắt buộc trong nhiều tiêu chuẩn bảo mật quốc tế (như ISO 27001, PCI DSS, HIPAA), giúp tổ chức tuân thủ pháp luật và xây dựng niềm tin với khách hàng.

1.3 CÁC TÍNH NĂNG MẬT MÃ TÍCH HỢP TRONG SQL SERVER

1.3.1 Tổng quan về tính năng mã hóa trong SQL Server

SQL Server tích hợp nhiều công cụ và phương pháp mã hóa bao gồm:

- Transparent Data Encryption (TDE): Mã hóa toàn bộ cơ sở dữ liệu ở cấp độ tệp (file-level encryption).
- Cell-Level Encryption: Mã hóa cụ thể từng cột chứa dữ liệu nhạy cảm.
- Always Encrypted: Mã hóa dữ liệu trên phía client, SQL Server không bao giờ thấy dữ liệu rõ ràng, thích hợp với dữ liệu cực kỳ nhạy cảm.
- Column-Level Encryption (CLE): Tính năng truyền thống, sử dụng các hàm EncryptByKey, DecryptByKey, EncryptByCert, v.v.
- Database Encryption Key (DEK), Service Master Key (SMK), Database Master Key (DMK): Hệ thống quản lý khóa bảo mật nội bộ của SQL Server.

1.3.2 So sánh các phương pháp mã hóa

Phương pháp	Mức mã hóa	Ưu điểm	Nhược điểm
TDE	Toàn bộ database	Dễ triển khai, không ảnh hưởng ứng dụng	Chỉ bảo vệ dữ liệu “at rest”, không mã hóa dữ liệu cụ thể
Column Encryption	Dữ liệu từng cột	Bảo vệ dữ liệu chi tiết, linh hoạt	Phức tạp, cần điều chỉnh truy vấn và mã hóa thủ công
Always Encrypted	Mã hóa phía client	Bảo mật cao nhất, ngay cả admin SQL cũng không thấy	Giới hạn tính năng SQL, khó triển khai cho dữ liệu lớn

1.3.3 Trường hợp sử dụng phù hợp của các phương pháp mã hóa

- TDE: Dành cho tổ chức muốn bảo vệ toàn bộ database với chi phí triển khai thấp, không thay đổi hệ thống hiện tại.
- Column Encryption: Phù hợp với hệ thống xử lý thông tin nhạy cảm như số CMND, tài khoản ngân hàng.
- Always Encrypted: Áp dụng trong môi trường yêu cầu mức độ bảo mật tối đa, ví dụ dữ liệu y tế, dữ liệu khách hàng trong ngân hàng.

CHƯƠNG 2: NGHIÊN CỨU CHI TIẾT CÁC THUẬT TOÁN MẬT

MÃ TRONG SQL SERVER

2.1. MÃ HÓA ĐỐI XỨNG (SYMMETRIC ENCRYPTION)

2.1.1 Giới thiệu và Nguyên tắc hoạt động

Mã hóa đối xứng là một phương pháp mật mã trong đó cùng một khóa được sử dụng cho cả hai quá trình: mã hóa và giải mã. Vì tính chất sử dụng chung một khóa, tốc độ mã hóa và giải mã trong phương pháp này thường rất nhanh, phù hợp với các hệ thống yêu cầu xử lý dữ liệu lớn. Trong SQL Server, mã hóa đối xứng thường được áp dụng để bảo vệ dữ liệu nhạy cảm trong các cột dữ liệu, chẳng hạn như số thẻ tín dụng, thông tin tài khoản ngân hàng, v.v.

2.1.2 Các thuật toán đối xứng phổ biến trong SQL Server

- **AES (Advanced Encryption Standard):**

- **Lịch sử phát triển:** Được chọn thay thế cho thuật toán DES vào năm 2001 bởi Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST).

- **Các chế độ hoạt động:**

- **ECB (Electronic Code Book):** Mỗi khối dữ liệu được mã hóa riêng biệt. Dễ bị phân tích mẫu, không được khuyến nghị sử dụng.

- **CBC (Cipher Block Chaining):** Mỗi khối mã hóa phụ thuộc vào khối trước đó, tăng cường bảo mật.

- **CTR (Counter Mode) và GCM (Galois/Counter Mode):** Hỗ trợ mã hóa song song, hiệu suất cao, thường dùng trong môi trường yêu cầu hiệu năng.

- **Độ dài khóa:** Hỗ trợ 128, 192 và 256 bit, trong đó AES-256 là chuẩn cao nhất về bảo mật.

- **DES và 3DES:**

- **Lịch sử:** DES là tiêu chuẩn mã hóa được sử dụng từ những năm 1970 nhưng dần bị thay thế do độ dài khóa chỉ 56 bit. 3DES là phiên bản nâng cấp, mã hóa ba lần với ba khóa khác nhau.
- **Nguyên tắc hoạt động:** 3DES tăng độ bảo mật bằng cách áp dụng ba lần mã hóa DES liên tiếp (EDE - Encrypt-Decrypt-Encrypt).
- **Lý do AES được ưu tiên hơn:** Hiệu suất cao hơn, bảo mật tốt hơn, không bị giới hạn bởi các cuộc tấn công hiện đại như brute-force.

- **Các thuật toán khác** (ít phổ biến hoặc ít được hỗ trợ): RC2, RC4 – tuy nhiên, hầu hết đã lỗi thời và không được khuyến nghị sử dụng trong các hệ thống hiện đại.

2.1.3 Ưu và nhược điểm của mã hóa đối xứng trong SQL Server

Ưu điểm	Nhược điểm
Tốc độ mã hóa, giải mã nhanh	Vấn đề chia sẻ và lưu trữ khóa an toàn
Dễ triển khai trong ứng dụng và SQL Server	Nếu khóa bị lộ, toàn bộ dữ liệu có thể bị giải mã
Hỗ trợ tốt trong SQL Server thông qua các hàm T-SQL	Khó kiểm soát truy cập chi tiết (so với bất đối xứng)

2.2. MÃ HÓA BẤT ĐỐI XỨNG (ASYMMETRIC ENCRYPTION)

2.2.1 Giới thiệu và nguyên tắc hoạt động

Mã hóa bất đối xứng sử dụng cặp khóa: **khóa công khai (public key)** để mã hóa và **khóa riêng tư (private key)** để giải mã. Ưu điểm nổi bật của

phương pháp này là khả năng phân phối khóa công khai một cách an toàn, trong khi vẫn đảm bảo dữ liệu được bảo vệ bởi khóa riêng tư không bị lộ.

2.2.2 Các thuật toán bất đối xứng trong SQL Server

RSA (Rivest–Shamir–Adleman): RSA sử dụng hai số nguyên rất lớn và các phép toán số học để tạo ra khóa công khai và khóa riêng tư.

- Quá trình tạo khóa: Hai số nguyên lớn được chọn, nhân lại để tạo thành khóa công khai và khóa riêng tư.

- Mã hóa và giải mã: Dữ liệu được mã hóa bằng khóa công khai và chỉ có thể giải mã bằng khóa riêng tư. -

ECC (Elliptic Curve Cryptography): ECC dựa trên các phép toán trên các đường cong elliptic, giúp tạo ra các khóa với độ dài nhỏ nhưng vẫn giữ được độ bảo mật cao.

- Ứng dụng trong mật mã: ECC đang dần trở thành lựa chọn phổ biến nhờ khả năng tạo khóa ngắn nhưng vẫn bảo mật cao.

- Ứng dụng của mã hóa bất đối xứng trong SQL Server để quản lý khóa và chữ ký số: Quản lý khóa là mã hóa bất đối xứng có thể được sử dụng để bảo vệ khóa đối xứng, chữ ký số sẽ làm cho dữ liệu có thể được ký bằng khóa riêng tư để xác thực nguồn gốc và tính toàn vẹn của dữ liệu.

2.2.3 Ưu và nhược điểm của mã hóa bất đối xứng trong SQL Server

Ưu điểm	Nhược điểm
Bảo mật cao hơn nhờ tách biệt khóa công và khóa riêng	Hiệu suất mã hóa/giải mã chậm hơn so với mã hóa đối xứng
Không cần chia sẻ khóa riêng tư	Phức tạp hơn khi triển khai và quản lý

Dễ tích hợp với các mô hình chứng thực hiện đại	Không phù hợp cho mã hóa khối lượng lớn dữ liệu
---	---

2.3 HÀM BĂM (CRYPTOGRAPHIC HASH FUNCTIONS)

2.3.1 Giới thiệu và thuộc tính

Hàm băm là một hàm mật mã một chiều, chuyển đổi dữ liệu đầu vào thành một chuỗi ký tự cố định (gọi là “hash” hoặc “digest”). Một số đặc tính quan trọng:

- **Tính một chiều (One-way):** Không thể đảo ngược từ giá trị băm để lấy lại dữ liệu ban đầu.
- **Kháng va chạm (Collision Resistance):** Rất khó (gần như không thể) để tìm hai đầu vào khác nhau cho ra cùng một giá trị băm.
- **Đầu ra cố định:** Cho dù dữ liệu đầu vào dài bao nhiêu, kết quả băm vẫn có độ dài cố định.

2.3.2 Các thuật toán băm phổ biến

- **SHA-2 (SHA-256, SHA-512):** SHA-2 cung cấp độ bảo mật cao, với các độ dài băm là 256 và 512 bit. Cấu trúc cơ bản và ứng dụng: SHA-2 là chuẩn phổ biến để xác thực tính toàn vẹn của dữ liệu
- **MD5 (Message Digest Algorithm 5):** MD5 là thuật toán băm cũ, với độ dài băm 128 bit. MD5 không còn an toàn vì dễ bị tấn công va chạm, nhưng vẫn được sử dụng trong một số ứng dụng cũ

2.3.3 Ứng dụng của hàm băm trong SQL Server

Đảm bảo tính toàn vẹn dữ liệu, hàm băm giúp đảm bảo rằng dữ liệu không bị thay đổi khi truyền tải hoặc lưu trữ. Để lưu trữ mật khẩu người dùng có thể được băm và lưu trữ để tăng cường bảo mật

2.4 QUẢN LÝ KHÓA MÃ HÓA TRONG SQL SERVER

2.4.1 Tầm quan trọng

Việc quản lý khóa là yếu tố cốt lõi trong hệ thống bảo mật. Ngay cả khi thuật toán mã hóa mạnh mẽ, nhưng nếu khóa bị rò rỉ, toàn bộ dữ liệu có thể bị giải mã. SQL Server cung cấp các cơ chế quản lý khóa riêng biệt và có thể phân tầng theo cấp độ hệ thống và cơ sở dữ liệu.

2.4.2 Các phương pháp quản lý khóa

- Service Master Key (SMK): Là khóa chính của SQL Server, được sử dụng để bảo vệ các khóa khác
- Database Master Key (DMK): Quản lý khóa ở mức cơ sở dữ liệu, giúp bảo vệ dữ liệu trong cơ sở dữ liệu.
- Certificates (Chứng chỉ): Sử dụng chứng chỉ để xác thực và mã hóa dữ liệu.
- Asymmetric Keys (Khóa bất đối xứng): Sử dụng khóa công khai và khóa riêng tư cho các phương thức mã hóa và giải mã dữ liệu.
- Extensible Key Management (EKM): Cho phép tích hợp với các hệ thống quản lý khóa ngoài

2.4.3 Các nguyên tắc và thực hành tốt nhất trong quản lý khóa

- Sử dụng các phương thức bảo vệ khóa mạnh mẽ như mã hóa khóa và lưu trữ khóa ở nơi an toàn.
- Quản lý và thay đổi khóa thường xuyên để giảm nguy cơ rò rỉ.
- Đảm bảo rằng chỉ các cá nhân hoặc hệ thống có quyền truy cập hợp lệ mới có thể sử dụng khóa

THUẬT TOÁN MẬT MÃ TRONG SQL SERVER

3.1 LỰA CHỌN NGÔN NGỮ LẬP TRÌNH VÀ CÔNG CỤ

Trong dự án này, việc xây dựng ứng dụng tương tác với SQL Server được thực hiện với sự lựa chọn cẩn thận về ngôn ngữ lập trình và các công cụ hỗ trợ.

Ứng dụng được phát triển chủ yếu bằng ngôn ngữ Python. Python được chọn vì tính linh hoạt, cú pháp rõ ràng, dễ đọc và sự phong phú của các thư viện hỗ trợ phát triển web và tương tác cơ sở dữ liệu. Các thư viện quan trọng được sử dụng bao gồm Flask, một framework web micro để xây dựng ứng dụng web; pyodbc, thư viện để kết nối và tương tác với SQL Server thông qua ODBC; hashlib, thư viện cung cấp các thuật toán băm (như MD5) để xử lý mật khẩu; uuid, thư viện để tạo các định danh duy nhất (UUID) cho các mục đích khác nhau (ví dụ: reset token); ezgmail, thư viện để gửi email (ví dụ: email đặt lại mật khẩu); secrets, thư viện để tạo các giá trị ngẫu nhiên an toàn (ví dụ: secret key cho Flask); và re, thư viện để làm việc với biểu thức chính quy (ví dụ: kiểm tra định dạng mật khẩu).

Mặc dù code không chỉ định rõ IDE được sử dụng, nhưng các lựa chọn phổ biến cho Python bao gồm PyCharm, một IDE mạnh mẽ cho Python với nhiều tính năng hỗ trợ phát triển web; Visual Studio Code, một trình soạn thảo mã nguồn linh hoạt và phổ biến với các tiện ích mở rộng Python; và các công cụ dòng lệnh, cho phép phát triển Python bằng các trình soạn thảo văn bản đơn giản.

Về các thư viện và framework hỗ trợ, ứng dụng sử dụng thư viện pyodbc để thiết lập kết nối và thực hiện các truy vấn đến SQL Server. Code định nghĩa hàm `get_connection()` để tạo kết nối đến cơ sở dữ liệu. Thư viện hashlib được sử dụng để thực hiện băm mật khẩu bằng thuật toán MD5, và code định nghĩa hàm `hash_md5()` để thực hiện thao tác băm. Thư viện secrets cũng được sử dụng để tạo `secret_key` cho Flask.

Tóm lại, ứng dụng được phát triển chủ yếu bằng Python với sự hỗ trợ của các thư viện và framework liên quan. Python được chọn vì tính linh hoạt và sự phong phú của các thư viện hỗ trợ. Ứng dụng sử dụng pyodbc để tương tác với SQL Server và hashlib để thực hiện băm mật khẩu. Cần lưu ý rằng báo cáo này chỉ tập trung vào các công cụ và thư viện được sử dụng trong code cung cấp, và có thể có các công cụ hoặc quy trình phát triển khác không được thể hiện rõ ràng trong code.

3.2 THIẾT KẾ CƠ SỞ DỮ LIỆU CHO ỨNG DỤNG

Ứng dụng này sử dụng cơ sở dữ liệu SQL Server để quản lý và lưu trữ thông tin người dùng cùng các dữ liệu liên quan. Thiết kế cơ sở dữ liệu đóng vai trò quan trọng trong việc đảm bảo tính toàn vẹn, bảo mật và hiệu quả của ứng dụng.

Cấu trúc cơ sở dữ liệu được định nghĩa xoay quanh bảng users, nơi lưu trữ thông tin tài khoản người dùng. Bảng này bao gồm các cột sau:

- username (VARCHAR(50), PRIMARY KEY): Đây là khóa chính của bảng, đảm bảo mỗi người dùng có một định danh duy nhất. Cột này lưu trữ tên đăng nhập của người dùng.
- combined_password_phone_hash (VARCHAR(255), NOT NULL): Cột này lưu trữ mã băm của mật khẩu người dùng, được kết hợp với số điện thoại. Điều quan trọng cần lưu ý là code sử dụng thuật toán MD5 để băm, một thuật toán băm một chiều, không phải là phương pháp mã hóa hai chiều.
- address (NVARCHAR(255)): Cột này lưu trữ địa chỉ của người dùng.
- phone (VARCHAR(20)): Cột này lưu trữ số điện thoại của người dùng.
- email (VARCHAR(100), NOT NULL): Cột này lưu trữ địa chỉ email của người dùng.

- `is_admin` (BIT, DEFAULT 0): Cột này xác định xem người dùng có quyền quản trị hay không, với giá trị mặc định là 0 (không có quyền quản trị).

Về phương pháp "mã hóa", cần nhấn mạnh rằng ứng dụng hiện tại không thực hiện mã hóa dữ liệu tùy ý. Thay vào đó, nó sử dụng kỹ thuật hashing để bảo vệ mật khẩu người dùng. Quá trình băm mật khẩu được thực hiện bằng thuật toán MD5, được triển khai trong hàm `hash_md5(text)` của `app.py`. Để tăng cường bảo mật, mật khẩu được kết hợp với số điện thoại trước khi băm, tạo ra một dạng "salt" đơn giản. Điều quan trọng là chỉ có mã băm của mật khẩu được lưu trữ trong cột `combined_password_phone_hash`, không phải mật khẩu gốc. Khi người dùng cố gắng đăng nhập, mật khẩu được nhập sẽ được băm lại và so sánh với mã băm đã lưu để xác thực.

Cần lưu ý rằng các thông tin người dùng khác, chẳng hạn như địa chỉ, số điện thoại và email, được lưu trữ trực tiếp trong cơ sở dữ liệu mà không áp dụng bất kỳ hình thức mã hóa nào. Do đó, cơ sở dữ liệu hiện tại không có các cột bổ sung nào được thiết kế để lưu trữ thông tin về thuật toán mã hóa hoặc khóa mã hóa.

Tóm lại, thiết kế cơ sở dữ liệu xoay quanh bảng `users` để quản lý thông tin người dùng. Mặc dù ứng dụng sử dụng hashing (MD5) để bảo vệ mật khẩu, nhưng nó không triển khai mã hóa cho các dữ liệu khác. Cấu trúc cơ sở dữ liệu phản ánh việc sử dụng hashing cho mật khẩu, không có các thành phần bổ sung để quản lý khóa hoặc thuật toán mã hóa cho các trường dữ liệu khác. Điều quan trọng là báo cáo này chỉ mô tả cấu trúc và phương pháp bảo vệ mật khẩu như được thể hiện trong code cung cấp, và các yêu cầu bảo mật nâng cao hơn có thể không được bao gồm.

3.3 TRIỂN KHAI MÃ HÓA VÀ GIẢI MÃ DỮ LIỆU

Mã hóa dữ liệu (Hashing):

- Code sử dụng thư viện `hashlib` của Python để thực hiện mã hóa (hashing) mật khẩu trước khi lưu trữ vào cơ sở dữ liệu. Cụ thể, hàm `hash_md5(text)` được định nghĩa để tạo ra giá trị hash MD5 của chuỗi đầu vào.

- Quá trình "mã hóa" ở đây thực chất là tạo ra một giá trị hash, một chuỗi ký tự đại diện cho dữ liệu gốc. MD5 là một thuật toán hash một chiều, nghĩa là không thể khôi phục dữ liệu gốc từ giá trị hash.
- Trong quá trình đăng ký và đăng nhập, mật khẩu của người dùng được kết hợp với số điện thoại, sau đó được hash bằng MD5. Giá trị hash này được lưu trữ trong cột `combined_password_phone_hash` của bảng `users`.

Giải mã dữ liệu:

- Về mặt kỹ thuật, với thuật toán hash MD5, không có quá trình "giải mã" thực sự. Thay vào đó, khi người dùng đăng nhập, mật khẩu họ nhập vào cũng được kết hợp với số điện thoại và hash bằng MD5. Giá trị hash kết quả được so sánh với giá trị hash đã lưu trong cơ sở dữ liệu. Nếu hai giá trị hash giống nhau, hệ thống xác thực rằng người dùng đã nhập đúng mật khẩu.
- Do tính chất một chiều của hàm hash, mật khẩu gốc không bao giờ được giải mã hoặc khôi phục, tăng cường bảo mật.

3.4 ỨNG DỤNG HÀM BẮM ĐỂ LƯU TRỮ MẬT KHẨU AN TOÀN

Trong phát triển ứng dụng web, việc bảo vệ mật khẩu của người dùng là một yêu cầu bảo mật cơ bản. Thay vì lưu trữ mật khẩu gốc trực tiếp vào cơ sở dữ liệu, phương pháp an toàn hơn là sử dụng các hàm băm. Hàm băm chuyển đổi mật khẩu thành một chuỗi ký tự có độ dài cố định, không thể đảo ngược, đảm bảo rằng ngay cả khi cơ sở dữ liệu bị xâm nhập, mật khẩu gốc vẫn được bảo vệ.

Trong code, việc băm mật khẩu được thực hiện bằng hàm `hash_md5(text)`, sử dụng thuật toán MD5 từ thư viện `hashlib` của Python. Hàm này nhận mật khẩu làm đầu vào và tạo ra một mã băm MD5, sau đó mã băm này được lưu trữ trong cơ sở dữ liệu. Ví dụ, khi người dùng đăng ký hoặc đăng nhập, mật khẩu được kết hợp với số điện thoại và sau đó được băm bằng `hash_md5` trước khi lưu hoặc so sánh.

Để tăng cường bảo mật, trong code đã áp dụng một hình thức "salt" đơn giản bằng cách kết hợp mật khẩu với số điện thoại trước khi băm. "Salt" là một chuỗi ngẫu nhiên được thêm vào mật khẩu để làm cho các cuộc tấn công

rainbow table trở nên khó khăn hơn. Tuy nhiên, việc sử dụng số điện thoại làm "salt" có thể có những hạn chế về tính dự đoán và tính ngẫu nhiên.

Điều quan trọng là cần nhận thức được rằng MD5 không còn được coi là một thuật toán băm an toàn cho việc lưu trữ mật khẩu do các vấn đề về va chạm. Do đó, code cần được cập nhật để sử dụng các thuật toán băm mạnh hơn. Các lựa chọn tốt hơn bao gồm SHA-256, SHA-512 hoặc Argon2, trong đó Argon2 được khuyến nghị là thuật toán băm mật khẩu mạnh nhất hiện nay.

Tóm lại, để lưu trữ mật khẩu an toàn, cần sử dụng các thuật toán băm mạnh, áp dụng salt ngẫu nhiên và duy nhất cho mỗi người dùng, và cần nhắc sử dụng các thư viện chuyên dụng để xử lý việc băm mật khẩu.

3.5 QUẢN LÝ KHÓA TRONG ỨNG DỤNG

Trong ứng dụng này, việc quản lý khóa là một yếu tố quan trọng trong việc bảo vệ thông tin nhạy cảm và đảm bảo an toàn cho các thao tác bảo mật. Tuy nhiên, cần lưu ý rằng code hiện tại chủ yếu tập trung vào việc băm mật khẩu, chứ không phải mã hóa dữ liệu tùy ý, do đó, các phương pháp quản lý khóa phức tạp không được triển khai.

Ứng dụng Flask sử dụng `app.secret_key` để ký các session của người dùng. Secret key này là một yếu tố quan trọng để duy trì tính toàn vẹn và bảo mật của phiên làm việc. Ứng dụng ưu tiên lấy `secret_key` từ biến môi trường `FLASK_SECRET_KEY`, đây là một phương pháp an toàn vì nó tránh việc lưu trữ trực tiếp khóa trong mã nguồn. Nếu biến môi trường `FLASK_SECRET_KEY` không được thiết lập, ứng dụng sẽ tạo một `secret_key` ngẫu nhiên. Tuy nhiên, trong môi trường production, việc tạo khóa ngẫu nhiên mỗi khi ứng dụng khởi động có thể tạo ra các rủi ro bảo mật. Việc sử dụng biến môi trường để lưu trữ `FLASK_SECRET_KEY` được coi là một thực hành tốt, và cần đảm bảo rằng biến môi trường này được thiết lập một cách an toàn trên máy chủ.

Ứng dụng cũng thực hiện "salt" cho mật khẩu bằng cách kết hợp mật khẩu với số điện thoại của người dùng trước khi băm bằng thuật toán MD5. Trong trường hợp này, số điện thoại đóng vai trò như một "salt" đơn giản. Số điện thoại được lưu trữ trực tiếp trong bảng users cùng với các thông tin khác của người dùng. Tuy nhiên, đây không phải là một phương pháp "quản lý khóa" theo nghĩa truyền thống, và việc sử dụng số điện thoại làm "salt" có thể có những hạn chế về bảo mật do tính dự đoán của số điện thoại.

Khi người dùng yêu cầu đặt lại mật khẩu, ứng dụng tạo một reset_token (UUID) và lưu trữ nó trong session của người dùng. Việc sử dụng UUID để tạo reset_token là phù hợp do tính duy nhất của UUID. Session cần được bảo mật để đảm bảo an toàn cho reset_token. Ứng dụng đã cấu hình một số tùy chọn bảo mật cho session. Tuy nhiên, ứng dụng hiện tại chưa có cơ chế hết hạn reset_token sau một khoảng thời gian nhất định, điều này có thể tạo ra rủi ro bảo mật.

Ngoài các phương pháp quản lý khóa cơ bản đã được triển khai, có nhiều phương pháp an toàn hơn có thể được sử dụng trong các ứng dụng phức tạp hơn. Chúng bao gồm sử dụng biến môi trường cho các khóa cấu hình nhỏ, lưu trữ khóa trong các tệp cấu hình được mã hóa, sử dụng các dịch vụ KMS chuyên dụng để quản lý khóa mã hóa, và sử dụng các mô-đun bảo mật phần cứng (HSM).

Tóm lại, ứng dụng hiện tại sử dụng một số phương pháp quản lý "khóa" cơ bản. Việc quản lý FLASK_SECRET_KEY bằng biến môi trường là một thực hành tốt. Tuy nhiên, cần cải thiện việc "salt" mật khẩu và bổ sung cơ chế hết hạn cho reset_token. Để tăng cường bảo mật, nên xem xét triển khai các phương pháp quản lý khóa an toàn hơn, đặc biệt là khi xử lý các dữ liệu nhạy cảm.

3.6 XÂY DỰNG GIAO DIỆN NGƯỜI DÙNG

Ứng dụng web được phát triển với một loạt các trang giao diện người dùng (UI) để tạo điều kiện cho sự tương tác giữa người dùng và hệ thống. Các trang này được xây dựng bằng cách sử dụng HTML và CSS để định hình cấu trúc và giao diện, kết hợp với Flask và Jinja, template engine của Flask, để xử lý logic phía máy chủ và hiển thị dữ liệu động.

Các trang giao diện chính bao gồm:

- Trang đăng ký (register.html): Trang này cho phép người dùng tạo tài khoản mới bằng cách thu thập thông tin như tên người dùng, mật khẩu, địa chỉ, số điện thoại và email. Giao diện bao gồm các trường nhập liệu để thu thập thông tin này và một nút gửi để thực hiện quá trình đăng ký. Ngoài ra, trang cũng cung cấp khả năng hiển thị thông báo lỗi để thông báo cho người dùng về bất kỳ vấn đề nào phát sinh trong quá trình đăng ký, chẳng hạn như tên người dùng đã tồn tại hoặc mật khẩu không đáp ứng các yêu cầu bảo mật.
- Trang đăng nhập (login.html): Trang này cung cấp giao diện cho người dùng đã có tài khoản để đăng nhập vào hệ thống bằng cách nhập tên người dùng và mật khẩu. Tương tự như trang đăng ký, trang đăng nhập cũng có khả năng hiển thị thông báo lỗi để thông báo cho người dùng về các vấn đề như tên người dùng hoặc mật khẩu không chính xác. Để thuận tiện cho người dùng, trang này cũng cung cấp các liên kết đến trang quên mật khẩu và trang đăng ký.
- Trang chủ (home.html): Sau khi người dùng đăng nhập thành công, trang chủ sẽ hiển thị thông tin cá nhân của họ, bao gồm tên người dùng, địa chỉ, số điện thoại và email. Trang này cũng cung cấp các liên kết cho phép người dùng thực hiện các hành động như đăng xuất khỏi hệ thống, thay đổi mật khẩu của họ và truy cập trang quản lý tài khoản nếu họ có quyền quản trị. Đối với người dùng có quyền quản trị, trang chủ có thể mở rộng để hiển thị danh sách người dùng khác trong hệ thống và cung cấp các tùy chọn để xem, chỉnh sửa hoặc xóa thông tin của họ.

- Trang quản lý tài khoản (admin_dashboard.html): Trang này được thiết kế dành riêng cho người dùng có quyền quản trị. Nó hiển thị một danh sách toàn diện tất cả người dùng trong hệ thống, cùng với thông tin cơ bản của họ. Trang này cung cấp các công cụ cho phép quản trị viên thực hiện các hành động quản lý tài khoản, chẳng hạn như sửa đổi thông tin tài khoản hoặc xóa tài khoản người dùng. Để ngăn chặn các hành động vô ý, trang này thường yêu cầu xác nhận trước khi thực hiện xóa tài khoản.
- Trang sửa tài khoản (edit_user.html): Trang này cho phép người dùng có quyền quản trị sửa đổi thông tin cá nhân của người dùng khác, bao gồm địa chỉ, số điện thoại và email. Trang này thường hiển thị các trường nhập liệu đã được điền sẵn với thông tin hiện tại của người dùng, giúp đơn giản hóa quá trình chỉnh sửa.
- Trang quên mật khẩu (forgot_password.html): Trang này cung cấp một cơ chế cho người dùng để lấy lại quyền truy cập vào tài khoản của họ nếu họ quên mật khẩu. Người dùng có thể nhập địa chỉ email của họ, và hệ thống sẽ gửi một email chứa liên kết để đặt lại mật khẩu.
- Trang đặt lại mật khẩu (reset_password.html): Sau khi người dùng nhấp vào liên kết đặt lại mật khẩu trong email, trang này sẽ cho phép họ nhập mật khẩu mới và xác nhận mật khẩu đó. Trang này thường bao gồm các biện pháp kiểm tra để đảm bảo rằng mật khẩu mới đáp ứng các yêu cầu bảo mật và mã đặt lại mật khẩu là hợp lệ.
- Trang đổi mật khẩu (change_password.html): Trang này cho phép người dùng thay đổi mật khẩu hiện tại của họ trong khi vẫn đăng nhập vào hệ thống. Để đảm bảo an toàn, trang này thường yêu cầu người dùng nhập mật khẩu hiện tại của họ, cũng như mật khẩu mới và xác nhận mật khẩu mới. Trang này cũng có thể bao gồm các biện pháp kiểm tra để đảm bảo rằng mật khẩu hiện tại được nhập chính xác và mật khẩu mới đáp ứng các yêu cầu bảo mật.

Về việc xử lý dữ liệu "mã hóa", cần nhấn mạnh rằng trong code hiện tại, dữ liệu được "mã hóa" thực tế là dữ liệu đã được băm (hashing), đặc biệt là mật khẩu. Các trang giao diện không hiển thị mật khẩu ở dạng chưa được băm; thay vào đó, mật khẩu được băm trước khi lưu trữ, và chỉ có mã băm được lưu trong cơ sở dữ liệu. Khi người dùng đăng nhập, mật khẩu họ nhập được băm lại và so sánh với mã băm đã lưu. Phương pháp này giúp tăng cường bảo mật mật khẩu của người dùng, ngay cả trong trường hợp cơ sở dữ liệu bị xâm phạm.

Cuối cùng, ứng dụng triển khai một hệ thống kiểm soát quyền truy cập cơ bản, chủ yếu dựa trên vai trò quản trị. Trang quản lý tài khoản và các trang sửa/xóa tài khoản chỉ có thể được truy cập bởi người dùng có thuộc tính `is_admin` được đặt thành 1. Điều này đảm bảo rằng chỉ những người dùng được ủy quyền mới có thể xem và sửa đổi thông tin của người dùng khác.

Tóm lại, ứng dụng cung cấp một bộ các trang giao diện người dùng toàn diện để cho phép người dùng thực hiện nhiều thao tác khác nhau, từ đăng ký và đăng nhập đến quản lý tài khoản và đặt lại mật khẩu. Giao diện được thiết kế cẩn thận để tránh hiển thị dữ liệu nhạy cảm ở dạng chưa được băm, và ứng dụng triển khai kiểm soát quyền truy cập cơ bản để bảo vệ thông tin người dùng. Cần lưu ý rằng báo cáo này chỉ tập trung vào các khía cạnh của giao diện người dùng được thể hiện trong code, và có thể có các khía cạnh khác không được đề cập.

CHƯƠNG 4: KIỂM THỬ, KHẮC PHỤC LỖI VÀ ĐÁNH GIÁ ỨNG DỤNG

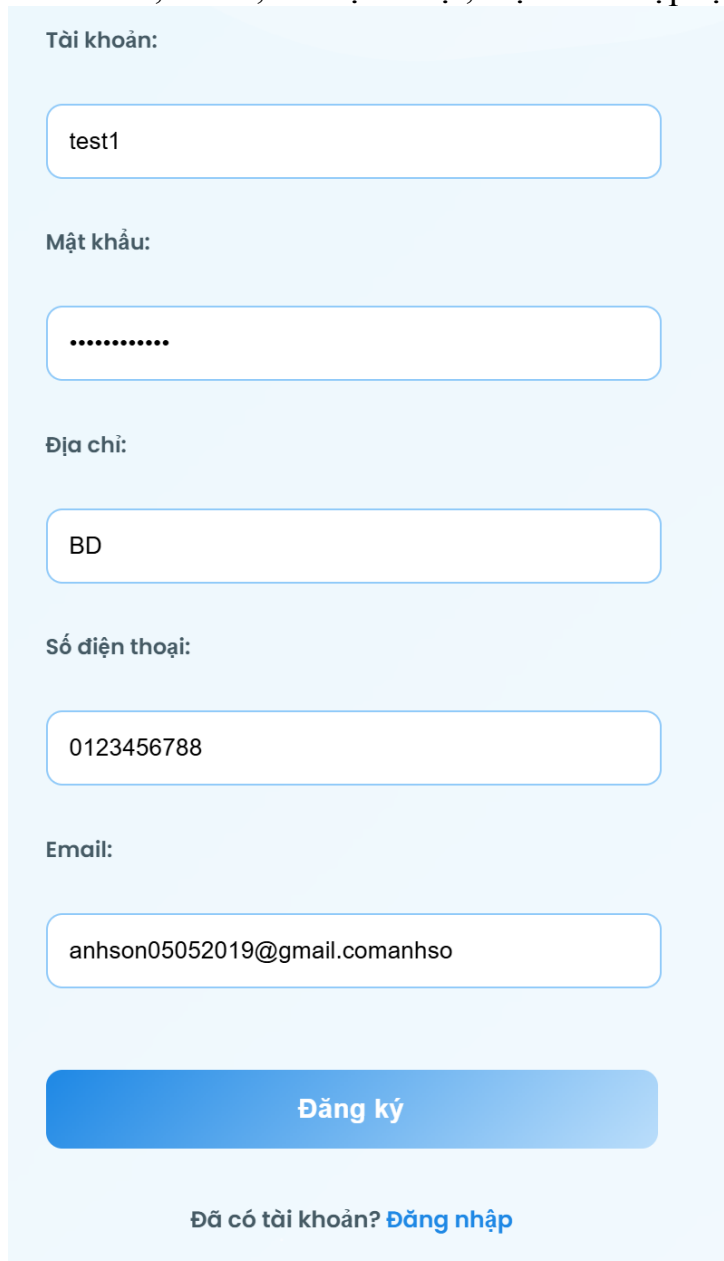
4.1 KIỂM THỬ

4.1.1 Kiểm thử chức năng

Đăng ký:

Kiểm tra thành công:

Đăng ký với username, email, số điện thoại, mật khẩu hợp lệ.

A registration form with a light blue background and rounded corners. It contains several input fields with blue borders and a blue 'Đăng ký' button. The fields are labeled 'Tài khoản:', 'Mật khẩu:', 'Địa chỉ:', 'Số điện thoại:', and 'Email:'. The 'Mật khẩu:' field uses dots for masking. At the bottom, there is a link 'Đã có tài khoản? Đăng nhập' in blue text.

Tài khoản:

Mật khẩu:

Địa chỉ:

Số điện thoại:

Email:

Đăng ký

Đã có tài khoản? [Đăng nhập](#)

Chào mừng, test1!

Đăng xuất
Đổi mật khẩu

Thông tin của bạn

Tên người dùng: test1
Địa chỉ: BD
Số điện thoại: 0123456788
Email: anhson05052019@gmail.comanhso

	username	combined_password_phone_nasn	address	phone	email	is_ad..
▶		782fdfa57ef016fa72e734ad26e7ce7a	Vinh Hoa, Phu Giao, Binh Duong	0123...	ann...	NULL
	admin	45a697baa9b39b12a1fc07c178187c62	Admin Address	0000...	admi...	True
	son	091e02f6fa1bbb10867bca61ac9ff282	Soos27 Vĩnh Hòa	0888...	nhok...	NULL
	sonle	091e02f6fa1bbb10867bca61ac9ff282	bd	0888...	nhok...	NULL
	test1	347d6dff457a73e7ed829650333612bb	BD	0123...	anhs...	False
*	NULL	NULL	NULL	NULL	NULL	NULL

Kiểm tra thất bại:
Username đã tồn tại.

Đăng ký

Tài khoản:

test1

Mật khẩu:

.....|

Địa chỉ:

bd

Số điện thoại:

0888103523

Email:

anhson05052018@gmail.com

Đăng ký

Đăng ký

Tài khoản đã tồn tại.

Tài khoản:

Email đã tồn tại.

Đăng ký

Tài khoản:

test3

Mật khẩu:

Địa chỉ:

bd

Số điện thoại:

0888103524

Email:

anhson05052019@gmail.com

Đăng ký

Email đã được sử dụng.

Tài khoản:

Số điện thoại đã tồn tại.

Đăng ký

Tài khoản:

test2

Mật khẩu:

Địa chỉ:

bd

Số điện thoại:

0888103523

Email:

anhson05052019@gmail.com

Đăng ký

Đăng ký

Số điện thoại đã được sử dụng.

Tài khoản:

Mật khẩu không đủ mạnh (ví dụ: quá ngắn, thiếu ký tự đặc biệt).

Tài khoản:

test4

Mật khẩu:

.

Địa chỉ:

bd

Số điện thoại:

0888103525

Email:

anhson05052020@gmail.com

Đăng ký

Mật khẩu phải có ít nhất 8 ký tự.

Tài khoản:

Số điện thoại không đúng định dạng (ví dụ: thiếu số).

Tài khoản:

test5

Mật khẩu:

.....|

Địa chỉ:

bd

Số điện thoại:

01

Email:

anhson05052020@gmail.com

Đăng ký

Đăng kí thành công, đã có lỗi xảy ra với sdt, cần được khắc phục

Đăng nhập:

Kiểm tra đăng nhập thành công với username/mật khẩu đúng.

Đăng nhập

Tài khoản:

test1

Mật khẩu:

.....|

Đăng nhập

Chưa có tài khoản? [Đăng ký](#) | [Quên mật khẩu?](#)

Chào mừng, test1!

[Đăng xuất](#) [Đổi mật khẩu](#)

Thông tin của bạn

Tên người dùng: test1

Địa chỉ: BD

Số điện thoại: 0123456788

Email: anhson05052019@gmail.comanhso

Kiểm tra đăng nhập thất bại với:
Sai username.

Đăng nhập

Tài khoản:

Mật khẩu:

Đăng nhập

Chưa có tài khoản? [Đăng ký](#) | [Quên mật khẩu?](#)

Đăng nhập

Tài khoản không tồn tại.

Tài khoản:

Sai mật khẩu.

Đăng nhập

Tài khoản không tồn tại.

Tài khoản:

Mật khẩu:

Đăng nhập

Chưa có tài khoản? [Đăng ký](#) | [Quên mật khẩu?](#)

Đăng nhập

Sai mật khẩu.

Tài khoản:

Mật khẩu:

Đăng nhập

Chưa có tài khoản? [Đăng ký](#) | [Quên mật khẩu?](#)

Quên mật khẩu:

Kiểm tra yêu cầu đặt lại mật khẩu thành công:

Email tồn tại trong hệ thống.

Email được gửi thành công.

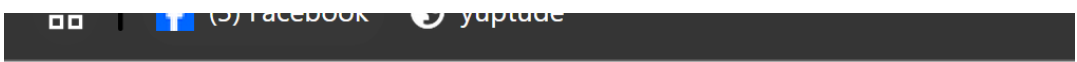
Quên Mật Khẩu

Email của bạn:

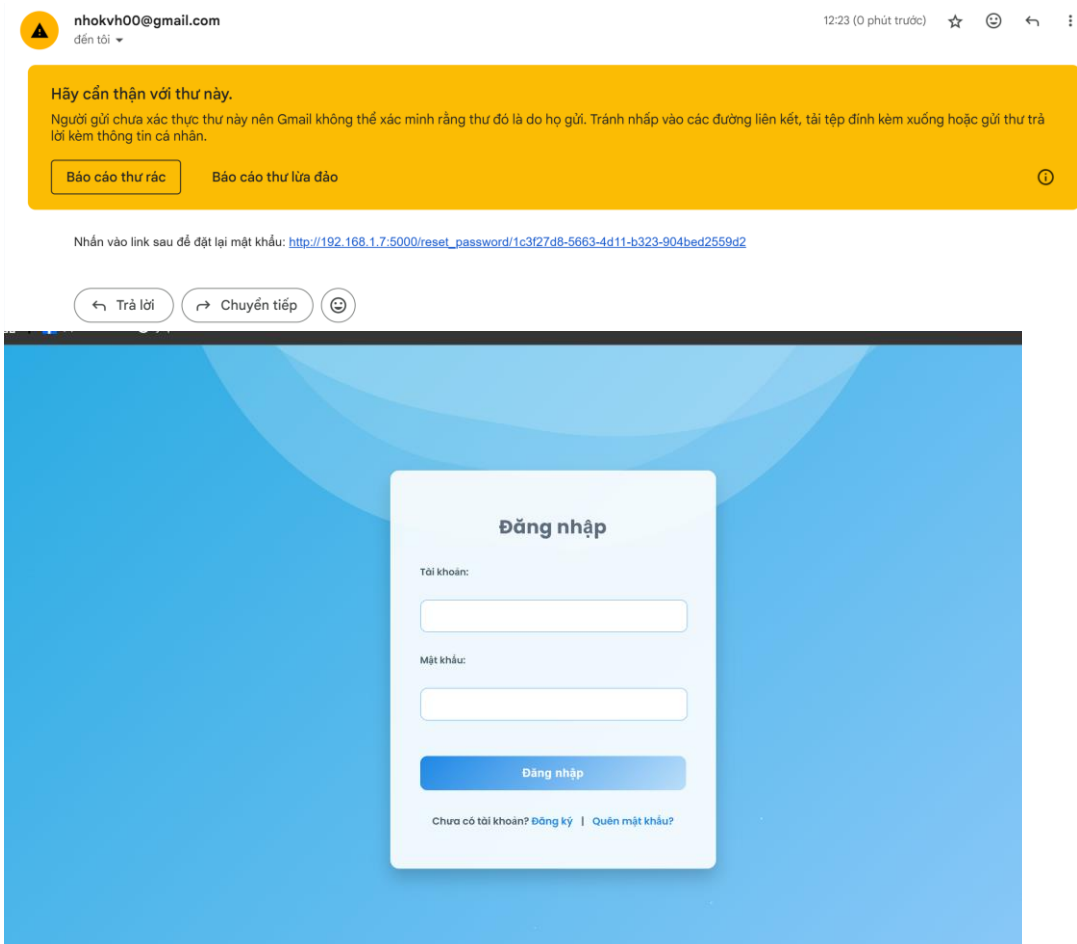
nhokvh00@gmail.com

Lấy lại mật khẩu

[Đăng nhập](#)



Một email chứa hướng dẫn đặt lại mật khẩu đã được gửi đến bạn.



Chức năng quên mk bị lỗi cần khắc phục
Kiểm tra yêu cầu đặt lại mật khẩu thất bại:
Email không tồn tại.

Quên Mật Khẩu

Email của bạn:

abcdd@gmail.com

Lấy lại mật khẩu

[Đăng nhập](#)

Quên Mật Khẩu

Không tìm thấy tài khoản với email này.

Email của bạn:

Lấy lại mật khẩu

[Đăng nhập](#)

Đổi mật khẩu:

Kiểm tra đổi mật khẩu thành công:

Mật khẩu hiện tại đúng.

Mật khẩu mới hợp lệ.

Đổi Mật Khẩu

Mật khẩu hiện tại:

Son11032005@



Mật khẩu mới:

- Ít nhất 8 ký tự
- Chứa ít nhất một chữ cái viết thường
- Chứa ít nhất một chữ cái viết hoa
- Chứa ít nhất một chữ số

.....

Xác nhận mật khẩu mới:

Son321614@



Đổi mật khẩu

[Quay lại trang chủ](#)

Chào mừng, son!

[Đăng xuất](#)

[Đổi mật khẩu](#)

Thông tin của bạn

Tên người dùng: son

Địa chỉ: Soos27 Vinh Hoa

Số điện thoại: 0888103523

Email: nhokvh00@gmail.com

Kiểm tra đổi mật khẩu thất bại:
Mật khẩu hiện tại sai.
Mật khẩu mới không hợp lệ.

Đổi Mật Khẩu

Mật khẩu mới và xác nhận mật khẩu không khớp.

Mật khẩu hiện tại:

Mật khẩu mới:

- Ít nhất 8 ký tự
- Chứa ít nhất một chữ cái viết thường
- Chứa ít nhất một chữ cái viết hoa
- Chứa ít nhất một chữ số

Xác nhận mật khẩu mới:

Đổi mật khẩu

[Quay lại trang chủ](#)

Quản lý người dùng (Admin):

Chào buổi chiều, admin!

[Đăng xuất](#)

[Đổi mật khẩu](#)

[Quản lý tài khoản](#)

12:42:17

Tên đăng nhập	Địa chỉ	Điện thoại	Email	Admin?	Hành động
	Vinh Hoa, Phu Giao, Binh Duong	0123456789	ann@gmail.com	Không	Sửa Xoá
admin	Admin Address	0000000000	admin@admin.com	Có	Sửa
son	Soos27 Vinh Hoa	0888103523	nhokvh00@gmail.com	Không	Sửa Xoá
sonle	bd	0888103523	nhokvh00@gmail.com	Không	Sửa Xoá
test1	BD	0123456788	anhson05052019@gmail.comanhso	Không	Sửa Xoá
test2	bd	0888103524	anhson05052019@gmail.com	Không	Sửa Xoá
test4	bd	0111111111	anhson05052020@gmail.com	Không	Sửa Xoá

Xem danh sách người dùng.

Sửa thông tin người dùng (địa chỉ, số điện thoại, email, quyền admin).

Sửa tài khoản: sonle

Địa chỉ:

Số điện thoại:

Email:

Quyền admin:

[Lưu thay đổi](#)

[← Quay lại trang quản lý](#)

Xóa người dùng.

Bạn chắc chắn muốn xóa tài khoản này?

[OK](#) [Huỷ](#)

Tên đăng nhập	Địa chỉ	Điện thoại	Email	Admin?	Hành động
Vinh Hoa, Phu Giao, Binh Duong	0123456789	ann@gmail.com	Không	Sửa Xóa	
admin	Admin Address	0000000000	admin@admin.com	Có	Sửa

Kiểm tra phân quyền:

Người dùng thông thường không được truy cập trang quản lý.
Admin có quyền truy cập và thực hiện các thao tác quản lý.

4.1.2 Kiểm thử bảo mật

Phân tích bảo mật:

Hashing mật khẩu:

- Đánh giá việc sử dụng MD5
- Mã nguồn sử dụng hàm `hash_md5` để băm mật khẩu. MD5 là một thuật toán băm cũ và được coi là không an toàn cho việc băm mật khẩu trong các ứng dụng hiện đại.
- Điểm yếu: MD5 dễ bị tấn công brute-force và collision attacks (tìm hai đầu vào khác nhau tạo ra cùng một giá trị băm). Điều này có nghĩa là kẻ tấn công có thể tìm ra mật khẩu gốc hoặc tạo ra một mật khẩu giả mạo có cùng giá trị băm với mật khẩu thật.
- Khuyến nghị: Nên thay thế MD5 bằng các thuật toán băm mạnh hơn như `bcrypt`, `scrypt`, hoặc `Argon2`. Các thuật toán này được thiết kế để chống lại các cuộc tấn công và làm chậm quá trình brute-force.

Quản lý phiên (session):

Đánh giá việc sử dụng session của Flask:

- Mã nguồn sử dụng session của Flask để quản lý phiên người dùng. Flask session sử dụng cookie để lưu trữ thông tin phiên ở phía client.
- Điểm cần lưu ý: Mặc định, Flask session cookie chỉ được ký (signed) chứ không được mã hóa (encrypted). Điều này có nghĩa là người dùng có thể xem nội dung của cookie (mặc dù không thể sửa đổi nếu không có secret key).
- Xem xét các thuộc tính bảo mật của cookie session:
- Mã nguồn không đặt các thuộc tính bảo mật quan trọng cho cookie session như `Secure` và `HttpOnly`.
- `Secure`: Thuộc tính `Secure` đảm bảo rằng cookie chỉ được gửi qua kết nối HTTPS. Điều này rất quan trọng để ngăn chặn việc đánh cắp cookie qua các kết nối không an toàn.

- HttpOnly: Thuộc tính HttpOnly ngăn chặn JavaScript phía client truy cập vào cookie. Điều này giúp giảm thiểu rủi ro tấn công XSS (Cross-Site Scripting).
- SameSite: Thuộc tính SameSite giúp bảo vệ ứng dụng khỏi các cuộc tấn công CSRF (Cross-Site Request Forgery).
- Khuyến nghị: Nên cấu hình Flask session để đặt các thuộc tính bảo mật này.

Xử lý lỗi:

- Đánh giá xem các thông báo lỗi có tiết lộ thông tin nhạy cảm không (ví dụ: thông tin về database).
- Qua quá trình kiểm tra thì việc thông báo lỗi không ảnh hưởng đến thông tin của mọi người
- Việc thực hiện xử lý lỗi kịp thời giúp ngăn chặn các tấn công về dữ liệu một cách hiệu quả

Quản lý khóa bí mật:

- Đánh giá việc lưu trữ và sử dụng secret_key của Flask.
- Việc quản lý secret_key trong Flask là một khía cạnh cực kỳ quan trọng của bảo mật ứng dụng. secret_key được Flask sử dụng để ký điện tử các cookie phiên (session cookies) và có thể được sử dụng bởi các extension khác để cung cấp tính bảo mật. Việc đánh giá cách lưu trữ và sử dụng secret_key là cần thiết để đảm bảo an toàn cho ứng dụng.
- Ở đây chúng ta sử dụng thư viện secrets của python để tạo ra chuỗi 32 bit ngẫu nhiên cho mỗi phiên hoạt động

Kết nối database:

- Đánh giá việc lưu trữ chuỗi kết nối đến SQL Server.

- Luôn ưu tiên sử dụng biến môi trường để lưu trữ các thành phần của chuỗi kết nối. Đây là một sự cân bằng tốt giữa tính bảo mật và sự tiện lợi cho nhiều ứng dụng.
- Đối với các ứng dụng quan trọng hoặc quy mô lớn, hãy cân nhắc sử dụng các hệ thống quản lý bí mật.
- Tuyệt đối không bao giờ hardcode chuỗi kết nối hoặc lưu nó ở những nơi không an toàn.
- Đảm bảo quyền truy cập vào server và các file cấu hình được kiểm soát chặt chẽ.
- Nếu bạn sử dụng file cấu hình, hãy đảm bảo nó nằm ngoài thư mục web root và chỉ có người dùng chạy ứng dụng mới có quyền đọc.
- Tránh commit các file cấu hình chứa thông tin nhạy cảm vào hệ thống kiểm soát phiên bản.

Kiểm thử tấn công (penetration testing) (ở mức cơ bản):

SQL Injection:

Thử nhập các ký tự đặc biệt vào các trường nhập liệu để phá vỡ câu lệnh SQL.

Đăng nhập

Tài khoản không tồn tại.

Tài khoản:

test1' UNION SELECT username, password FROM ai

Mật khẩu:

.....

Đăng nhập

Brute-Force:

- Thử đăng nhập nhiều lần với các mật khẩu khác nhau để xem có bị khóa tài khoản không.
- Tấn công thất bại

4.2 KHẮC PHỤC LỖI

4.2.1 Phân tích lỗi

- Lỗi đăng kí thành công khi số điện thoại không hợp lệ do quên đặt điều kiện khi thiết lập mật khẩu.
- Lỗi quên mật khẩu không thể đổi, do khi thực hiện code chèn sai đoạn mã của login.html

4.2.2 Sửa lỗi

Sửa lỗi số điện thoại khi đăng kí

```
def edit_user(username):
    """Admin edit user page."""
    if 'username' not in session:
        return redirect(url_for('login'))

    accounts = load_accounts()
    current_user = accounts.get(session['username'])
    if not current_user or not current_user.get('is_admin'):
        return "Không có quyền truy cập", 403

    if username not in accounts:
        return "Tài khoản không tồn tại", 404

    error = None
    if request.method == 'POST':
        address = request.form['address']
        phone = request.form['phone']
        email = request.form['email']
        is_admin = int(request.form.get('is_admin', 0))

        if not is_phone_valid(phone):
            error = "Số điện thoại phải là 10 số."
        elif error is None:
            accounts[username]['address'] = address
            accounts[username]['phone'] = phone
            accounts[username]['email'] = email
            accounts[username]['is_admin'] = is_admin
            save_accounts(accounts)
            return redirect(url_for('admin_dashboard'))
        else:
            return render_template('edit_user.html', username=username, data=accounts[username], error=error)

    return render_template('edit_user.html', username=username, data=accounts[username], error=error)
```

Sửa lỗi quên mật khẩu

Đoạn code đã được sửa lại
Thay login.html thành forgot password.html

```
def forgot_password():
    """Forgot password page."""
    error = None
    if request.method == 'POST':
        email = request.form['email']
        accounts = load_accounts()
        user_account = None
        for acc in accounts.items(): # Changed accounts.values() to accounts.items()
            if acc[1]['email'] == email: # acc is a tuple (username, account_data)
                user_account = acc
                break
        if user_account:
            reset_token = str(uuid.uuid4())
            session['reset_token'] = reset_token
            session['reset_email'] = email
            session['reset_username'] = user_account[0] # Store username
            try:
                ezgmail.send(
                    email,
                    "Yêu cầu đặt lại mật khẩu",
                    f"Nhấn vào link sau để đặt lại mật khẩu: {url_for('reset_password', reset_token=reset_token, _external=True)}"
                )
                return "Một email chứa hướng dẫn đặt lại mật khẩu đã được gửi đến bạn."
            except Exception as e:
                error = f"Không thể gửi email. Lỗi: {e}"
                print(e)
        else:
            error = "Không tìm thấy tài khoản với email này."
    return render_template('forgot_password.html', error=error)
```

4.2.3 Kiểm tra lại

Thực hiện lại kịch bản kiểm thử đã phát hiện lỗi để xác nhận lỗi đã được sửa.

Đăng ký

Số điện thoại phải là 10 số.

Tài khoản:

Mật khẩu:

Địa chỉ:

Số điện thoại:

Email:

Lỗi quên mật khẩu

Quên Mật Khẩu

Email của bạn:

nhokvh00@gmail.com

Lấy lại mật khẩu

[Đăng nhập](#)

Một email chứa hướng dẫn đặt lại mật khẩu đã được gửi đến bạn.

Đặt lại mật khẩu

Mật khẩu mới:

- Ít nhất 8 ký tự
- Chứa ít nhất một chữ cái viết thường
- Chứa ít nhất một chữ cái viết hoa
- Chứa ít nhất một chữ số

Xác nhận mật khẩu mới:

Đặt lại mật khẩu

[Quay lại trang chủ](#)

Đặt lại mật khẩu thành công!

Mật khẩu tài khoản của bạn đã được đặt lại thành công.

Bạn có thể đăng nhập bằng mật khẩu mới của mình.

Đăng nhập

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 TÓM TẮT KẾT QUẢ NGHIÊN CỨU VÀ PHÁT TRIỂN ỨNG DỤNG

Nghiên cứu này hướng đến việc xây dựng một ứng dụng quản lý tài khoản người dùng cơ bản, cung cấp các chức năng thiết yếu như đăng ký tài khoản mới, đăng nhập vào hệ thống, đăng xuất khỏi phiên làm việc, khôi phục mật khẩu khi quên và thay đổi mật khẩu đã thiết lập. Mục tiêu đầu tiên là hiện thực hóa đầy đủ các quy trình nghiệp vụ này thông qua giao diện người dùng trực quan và dễ sử dụng.

Song song với việc xây dựng chức năng, một mục tiêu quan trọng khác là triển khai các biện pháp bảo mật nền tảng nhằm bảo vệ thông tin người dùng. Nghiên cứu sẽ tập trung vào việc áp dụng kỹ thuật băm mật khẩu, cụ thể là sử dụng thuật toán MD5 kết hợp với số điện thoại duy nhất của người dùng trước khi lưu trữ vào cơ sở dữ liệu. Bên cạnh đó, việc quản lý phiên người dùng sau khi đăng nhập và thiết lập các thuộc tính bảo mật cơ bản cho cookie cũng sẽ được nghiên cứu và triển khai để đảm bảo an toàn cho phiên làm việc.

Để hỗ trợ người dùng trong trường hợp quên mật khẩu, nghiên cứu này cũng đặt mục tiêu tìm hiểu và tích hợp thư viện ezgmail của Python. Thư viện này sẽ được sử dụng để gửi email chứa liên kết đặt lại mật khẩu đến địa chỉ email đã đăng ký của người dùng, tạo điều kiện thuận lợi cho việc khôi phục quyền truy cập tài khoản.

Ngoài các chức năng dành cho người dùng thông thường, nghiên cứu còn hướng đến việc xây dựng các tính năng quản trị viên cơ bản. Các chức năng này bao gồm khả năng xem danh sách tất cả người dùng đã đăng ký, chỉnh sửa các thông tin cơ bản của người dùng (ngoại trừ mật khẩu vì lý do bảo mật), và xóa tài khoản người dùng không còn hoạt động hoặc vi phạm chính sách.

Cuối cùng, một mục tiêu mang tính nền tảng và học thuật quan trọng của nghiên cứu này là đi sâu vào cách thức hoạt động của hệ quản trị cơ sở dữ liệu SQL Server. Việc tìm hiểu về kiến trúc tổng quan, quy trình xử lý các truy vấn dữ liệu, cơ chế lưu trữ thông tin và các khía cạnh bảo mật cơ bản mà SQL Server cung cấp sẽ giúp xây dựng một mô hình đơn giản. Mô hình này sẽ mô phỏng một số hành vi cốt lõi của SQL Server trong việc quản lý dữ liệu người dùng, từ đó làm sáng tỏ quá trình lưu trữ và truy xuất dữ liệu trong bối cảnh ứng dụng đang phát triển. Việc này không chỉ củng cố kiến thức về quản lý cơ sở dữ liệu mà còn tạo nền tảng cho việc tối ưu hóa và bảo mật ứng dụng trong tương lai.

5.2 THẢO LUẬN VỀ NHỮNG THÁCH THỨC VÀ BÀI HỌC KINH NGHIỆM TRONG QUÁ TRÌNH THỰC HÀNH

Quá trình nghiên cứu và phát triển ứng dụng quản lý tài khoản đã đối diện với một số thách thức đáng chú ý.

Một trong những thách thức lớn nhất là việc đảm bảo an ninh cho dữ liệu người dùng. Việc lựa chọn thuật toán băm MD5, dù đơn giản trong triển khai, đã bộc lộ những hạn chế về khả năng chống lại các phương pháp tấn công hiện đại.

Thêm vào đó, việc xử lý các lỗi tiềm ẩn và ngoại lệ trong quá trình kết nối cơ sở dữ liệu, gửi email hay tương tác với người dùng đòi hỏi sự cẩn trọng và bao quát để đảm bảo tính ổn định của ứng dụng.

Tương tác hiệu quả với hệ quản trị cơ sở dữ liệu SQL Server, từ việc thiết lập kết nối ổn định đến việc xây dựng các truy vấn SQL tối ưu và an toàn, cũng là một khía cạnh đòi hỏi kiến thức và kinh nghiệm.

Quá trình tìm hiểu và áp dụng các cơ chế quản lý phiên và cookie một cách an toàn cũng đặt ra những yêu cầu về mặt kỹ thuật. Từ những thách thức này, nhiều bài học kinh nghiệm quý báu đã được rút ra. Ưu tiên hàng đầu trong phát triển ứng dụng cần đặt vào bảo mật, đòi hỏi việc lựa chọn các thuật toán mã hóa và băm mạnh mẽ hơn. Việc xử lý lỗi một cách toàn diện và dự đoán các tình huống ngoại lệ là yếu tố then chốt để xây dựng một ứng dụng đáng tin cậy.

Bên cạnh đó, việc kiểm tra và xác thực dữ liệu đầu vào từ người dùng một cách nghiêm ngặt là cần thiết để ngăn chặn các vấn đề về dữ liệu không hợp lệ và các nguy cơ bảo mật tiềm ẩn.

Cuối cùng, việc tiếp cận phát triển theo từng module nhỏ và tiến hành kiểm thử kỹ lưỡng từng phần đã chứng minh là một phương pháp hiệu quả để phát hiện và khắc phục lỗi sớm trong quá trình phát triển.

5.3 ĐỀ XUẤT CÁC HƯỚNG PHÁT TRIỂN TIẾP THEO CHO ỨNG DỤNG

Để nâng cao chất lượng và tính bảo mật của ứng dụng quản lý tài khoản, một số hướng phát triển tiềm năng có thể được xem xét trong tương lai.

Về mặt bảo mật, việc nâng cấp thuật toán băm mật khẩu lên các tiêu chuẩn mạnh mẽ hơn như SHA-256 hoặc Argon2, kết hợp với việc sử dụng salt ngẫu nhiên cho mỗi mật khẩu, là một bước quan trọng.

Triển khai giao thức HTTPS để mã hóa toàn bộ dữ liệu trao đổi giữa người dùng và máy chủ sẽ giúp ngăn chặn các cuộc tấn công nghe lén.

Bên cạnh đó, việc tích hợp cơ chế xác thực hai yếu tố (2FA) sẽ cung cấp một lớp bảo vệ bổ sung cho tài khoản người dùng.

Về hiệu suất, việc tối ưu hóa các truy vấn cơ sở dữ liệu và xem xét sử dụng các kỹ thuật bộ nhớ đệm có thể cải thiện đáng kể tốc độ phản hồi của ứng dụng, đặc biệt khi có lượng lớn người dùng.

Để quản lý và giám sát hệ thống tốt hơn, việc triển khai hệ thống ghi log chi tiết các hoạt động của người dùng và quản trị viên, cũng như các sự kiện bảo mật, sẽ rất hữu ích.

Ngoài ra, việc nghiên cứu và tích hợp thêm các thuật toán mật mã tiên tiến khác cho việc bảo vệ dữ liệu nhạy cảm và xây dựng các API bảo mật cho phép các ứng dụng khác tương tác an toàn với hệ thống quản lý tài khoản cũng là những hướng phát triển đáng cân nhắc.

Cuối cùng, việc nâng cấp giao diện quản trị để cung cấp nhiều công cụ và thông tin hơn cho việc quản lý người dùng sẽ giúp tăng cường khả năng quản trị hệ thống.

TÀI LIỆU THAM KHẢO

- [1] Microsoft Corporation, “Encryption in SQL Server,” 2023. [Online]. Available: <https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/sql-server-encryption>
- [2] National Institute of Standards and Technology (NIST), “FIPS PUB 197: Advanced Encryption Standard (AES),” Nov. 2001. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>
- [3] W. Stallings, Cryptography and Network Security: Principles and Practice, 8th ed., Pearson Education, 2022.
- [4] Microsoft Docs, “Always Encrypted (Database Engine),” 2022. [Online]. Available: <https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/always-encrypted-database-engine>
- [5] C. Paar and J. Pelzl, Understanding Cryptography: A Textbook for Students and Practitioners, Springer, 2010.
- [6] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, Handbook of Applied Cryptography, CRC Press, 1996.
- [7] International Organization for Standardization, “ISO/IEC 27001:2022 – Information Security Management Systems – Requirements,” 2022.
- [8] D. R. Stinson and M. B. Paterson, Cryptography: Theory and Practice, 4th ed., CRC Press, 2018.
- [9] R. Anderson, Security Engineering: A Guide to Building Dependable Distributed Systems, 3rd ed., Wiley, 2020.
- [10] S. William and L. Brown, Applied Cryptography: Protocols, Algorithms, and Source Code in C, 2nd ed., Wiley, 1996.

DANH SÁCH THÀNH VIÊN THAM GIA ĐỒ ÁN CUỐI KÌ

Nhóm: 10d (Lớp: 242CRYP331980_02)

Tên đề tài: Nghiên cứu các thuật toán mật mã được ứng dụng trong hệ quản trị cơ sở dữ liệu Microsoft SQL Server

STT	HỌ VÀ TÊN SINH VIÊN	MSSV	% HOÀN THÀNH
1	Nguyễn Vĩnh An	23162002	100%
2	Phạm Gia Bảo	23162007	100%
3	Lê Thanh Sơn	23162084	100%
4	Nguyễn Văn Tâm	23162088	100%

Ghi chú:

Tỷ lệ %= 100%: Mức độ phần trăm hoàn thành của từng sinh viên tham gia

Nhóm trưởng: Lê Thanh Sơn SĐT: 0888103523

Email: 23162084@student.hcmute.edu.vn

Điểm số:.....

Nhận xét giáo viên:

.....
.....
.....
.....
.....

Thủ Đức, ngày tháng năm 2025

Ký xác nhận giáo viên